



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G5

System Integration and Architecture

System Design Document (SDD)

Project Title: SplitShare

Prepared By: Caones, Lerah A.

Version: 2

Date: February 20, 2026

Status: Draft

REVISION HISTORY TABLE

Versio n	Date	Author	Changes Made	Status
0.1	02/20/2026	Lerah A. Caones	Initial draft	Draft
0.2	02/27/2026	Lerah A. Caones	Added API specifications	Review
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

SplitShare is a specialized financial tracking application designed for web and mobile platforms. It addresses the common friction found in shared living or group travel by providing a transparent, real-time "Who owes Who" ledger. Unlike complex accounting software, SplitShare focuses on a minimalist user experience that allows for quick expense logging, receipt verification, and debt settlement.

1.2 Objectives

1. Develop a cross-platform system (Spring Boot backend, React web, Android mobile) for real-time expense sharing.
2. Implement a robust "Settle Up" engine that accurately reconciles debts between multiple users within a group.
3. Provide a secure repository for financial transparency through digital receipt uploads and transactional history.
4. Ensure a consistent user experience with a "mobile-first" design approach for users logging expenses on the go.

1.3 Scope

Included Features:

- Secure User Authentication (JWT-based).
- Group Management (Create/Join groups for roommates, trips, etc.).
- Expense Logging with categorical tagging.
- **Receipt Upload:** Image attachment for every transaction.
- **Live Balance Sheet:** Automated calculation of individual net balances.
- **Settle Up Function:** Formalized debt payment recording.
- Personal Activity Feed and Profile management.

Excluded Features:

- Direct bank account integration (Open Banking).
- Automatic currency conversion (fixed to one currency per group).
- In-app chat or messaging system.

- Splitting by percentages or shares (initially limited to equal splits).

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the **SplitShare** system. It details the architecture, functional behavior, and technical requirements to ensure seamless integration between the Spring Boot API, the React web interface, and the Android mobile application.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: SplitShare

Domain: Personal Finance / Productivity

Primary Users:

1. Roommates
2. Travelers
3. Friends

Problem Statement: Manual tracking of shared costs often leads to disputes, forgotten debts, and a lack of transparency regarding who has paid for what.

Solution: A centralized ledger that provides an instant "Net Balance" and a verified trail of receipts.

2.2 Core User Journeys

Journey 1: Group Expense Creation

1. User logs in and selects the "Europe Trip" group.
2. User clicks "Add Expense" and enters "\$120 for Dinner."
3. User uploads a photo of the restaurant receipt.
4. The system automatically splits the cost and updates the "Who owes Who" list for all group members.

Journey 2: Settling a Debt

1. User views their dashboard and sees they owe a roommate \$50.

2. User pays the roommate via an external method (cash).
3. User clicks "Settle Up" in SplitShare.
4. The system records the payment and resets the balance between those two users to zero.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User Auth
2. Group Dashboard
3. Expense Logging
4. Settle Up Transaction
5. Net Balance Calculation

SHOULD HAVE

1. Receipt Image Upload
2. Profile History
3. Searchable Transactions
4. User Social Connections (follow/unfollow)
5. Google OAuth2 Login
6. Admin Audit Logs

COULD HAVE

1. Export to CSV
2. Category-based spending charts
3. Push notifications for new expenses

WON'T HAVE

1. Real-world payment gateway
2. Multi-currency support

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Profile Setup.
- **Fields:** Email, Password, Name, Profile Picture (optional).
- **Validation:** Email must be unique; Passwords must be at least 8 characters.
- **Security:** JWT (JSON Web Tokens) for session management; passwords hashed via Bcrypt. Google OAuth2 supported as an alternative login method, auto-creating user accounts on first sign-in.

- **API Endpoints:** POST /auth/register, POST /auth/login, GET /oauth2/authorization/google

Feature: Group Dashboard

- **Screens:** Group Home, Balance Detail.
- **Display:** Total Group Spending, Personal Net Balance (Green for "Owed", Red for "Owe").
- **Logic:** The system sums all expenses where the user is a participant and subtracts all expenses the user has paid for.
- **API Endpoints:** GET /groups/{groupId}

Feature: Expense Logging & Receipt Upload

- **Screens:** Add Expense Form, Receipt Preview.
- **Fields:** Amount, Description, Category (Food, Rent, etc.), Date, Paid By (User), Split With (All or Specific members).
- **File Handling:** Multipart/form-data for image uploads. Supports JPG and PNG.
- **API Endpoints:** POST /groups/{groupId}/expenses (multipart/form-data — receipt image included in the same request), PUT /groups/{groupId}/expenses/{expenseId} (update with optional new receipt), DELETE /groups/{groupId}/expenses/{expenseId}

Feature: Settle Up Transaction

- **Logic:** The "Settle Up" function is a **Manual Reconciliation** tool. It does not trigger a financial transfer. Instead, it creates a settlement record in the database that offsets the debt between two users.
- **User Workflow:** 1. User A pays User B in real life (Cash, GCash, etc.). 2. User B confirms the settlement from their dashboard, setting confirmedByUserB = true. 3. Only when both users have confirmed is the debt considered resolved and the balance returns to \$0.
- **API Endpoint:** POST /groups/{groupId}/settlements

Feature: Personal Profile & History

- **Screens:** User Profile, Global Activity Feed.
- **Functions:** View "All Time" spending, manage account settings, and see a chronological list of all transactions across all groups.
- **API Endpoints:** GET /users/me/history

Feature: User Social Connections

- **Description:** Users can follow each other to establish trusted connections, making it easier to add members to groups.
- **Functions:** Search users by name or email. Follow/Unfollow. View mutual connections (users who follow each other).
- **API Endpoints:** GET /users/search?q=, POST /users/{id}/follow, DELETE /users/{id}/follow, GET /users/me/followers, GET /users/me/following, GET /users/mutuals

Feature: Admin Management Console

- **Screens:** Admin Dashboard, User Management List, System Audit Log.
- **Functions:** * **User Suspension:** Ability to deactivate accounts that violate terms or are flagged for fraudulent receipt uploads.
 - **Group Overview:** View all active groups and their total volume of transactions to monitor system load.
- **Access Control:** This feature is strictly restricted to users with the ROLE_ADMIN flag in the database.
- **API Endpoints:** GET /admin/users, PUT /admin/users/{id}/status, GET /admin/analytics

2.5 Acceptance Criteria

AC-1: Real-time Balance Update

Given I am in a "Roommates" group with a \$0 balance.

When I add a \$100 expense for "Groceries" split equally with 1 other person.

Then the Dashboard should immediately show I am "Owed \$50" and the other person "Owes \$50."

AC-2: Valid Receipt Attachment

Given I am logging a new expense.

When I upload a photo of my receipt and save.

Then the image should be viewable by all group members when they click on that specific transaction.

AC-3: Successful Settlement

Given I owe my friend \$25.

When I click "Settle Up" and confirm the payment.

Then my "Net Balance" with that specific friend should return to \$0.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- **API Response Time:** $\leq 500\text{ms}$ for balance calculations to ensure the dashboard feels instantaneous.
- **Concurrency:** Support up to 100 concurrent users logging expenses across different groups.
- **Image Processing:** Receipt uploads must be compressed to $< 1\text{MB}$ before cloud storage to maintain mobile performance.

3.2 Security Requirements

- **Data Integrity:** All "Settle Up" transactions must be atomic; if one part of the database update fails, the entire transaction rolls back.
- **Authentication:** Mandatory JWT token validation for every request except login and registration.
- **Authorization:** Strict Role-Based Access Control (RBAC) ensuring only Admins can access user suspension tools and only group members can view their specific group's receipts.

3.3 Compatibility Requirements

- **Mobile:** Native Android support for API Level 24+ (Android 7.0+).
- **Web:** Responsive design optimized for Chrome, Safari, and Firefox.
- **Screen Sizes:** Adaptive layouts for Mobile (360px), Tablet (768px), and Desktop (1024px).

3.4 Usability Requirements

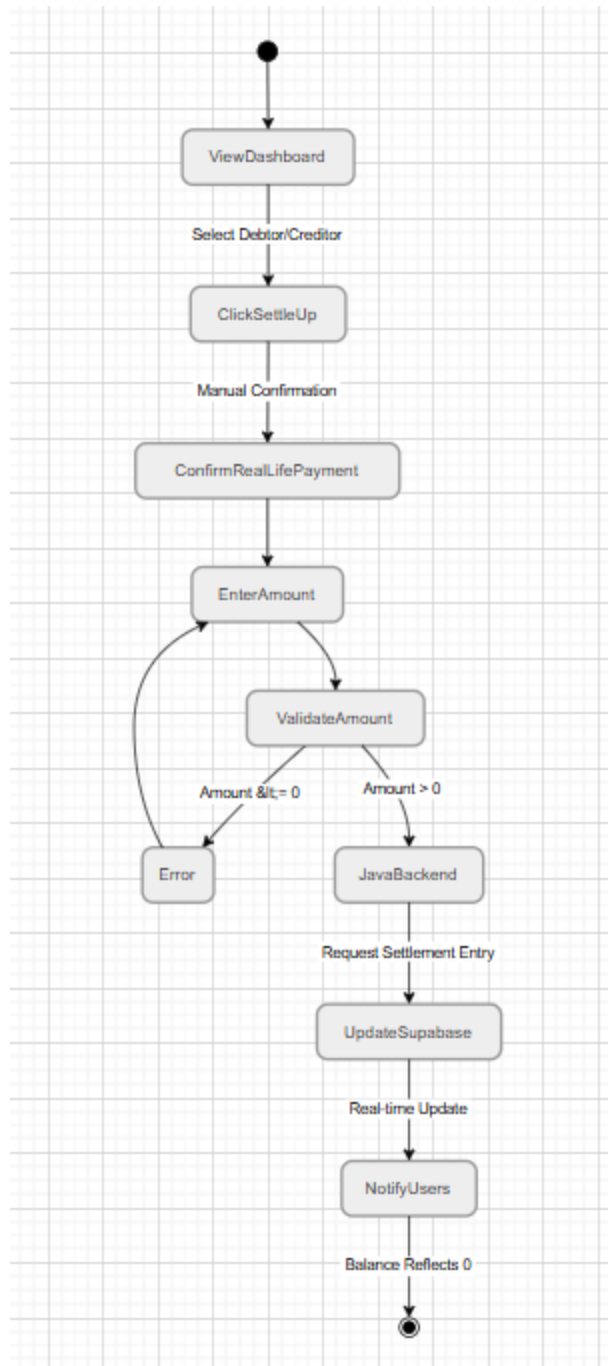
- **First Entry Efficiency:** A new user should be able to create a group and log their first shared expense in under **3 minutes**.
- **Accessibility: WCAG 2.1 Level AA** compliance for the web application to ensure high contrast for financial figures and screen-reader support.
- **Intuitive Navigation:** Consistent sidebar/bottom navigation across web and mobile platforms to allow users to switch between groups seamlessly.

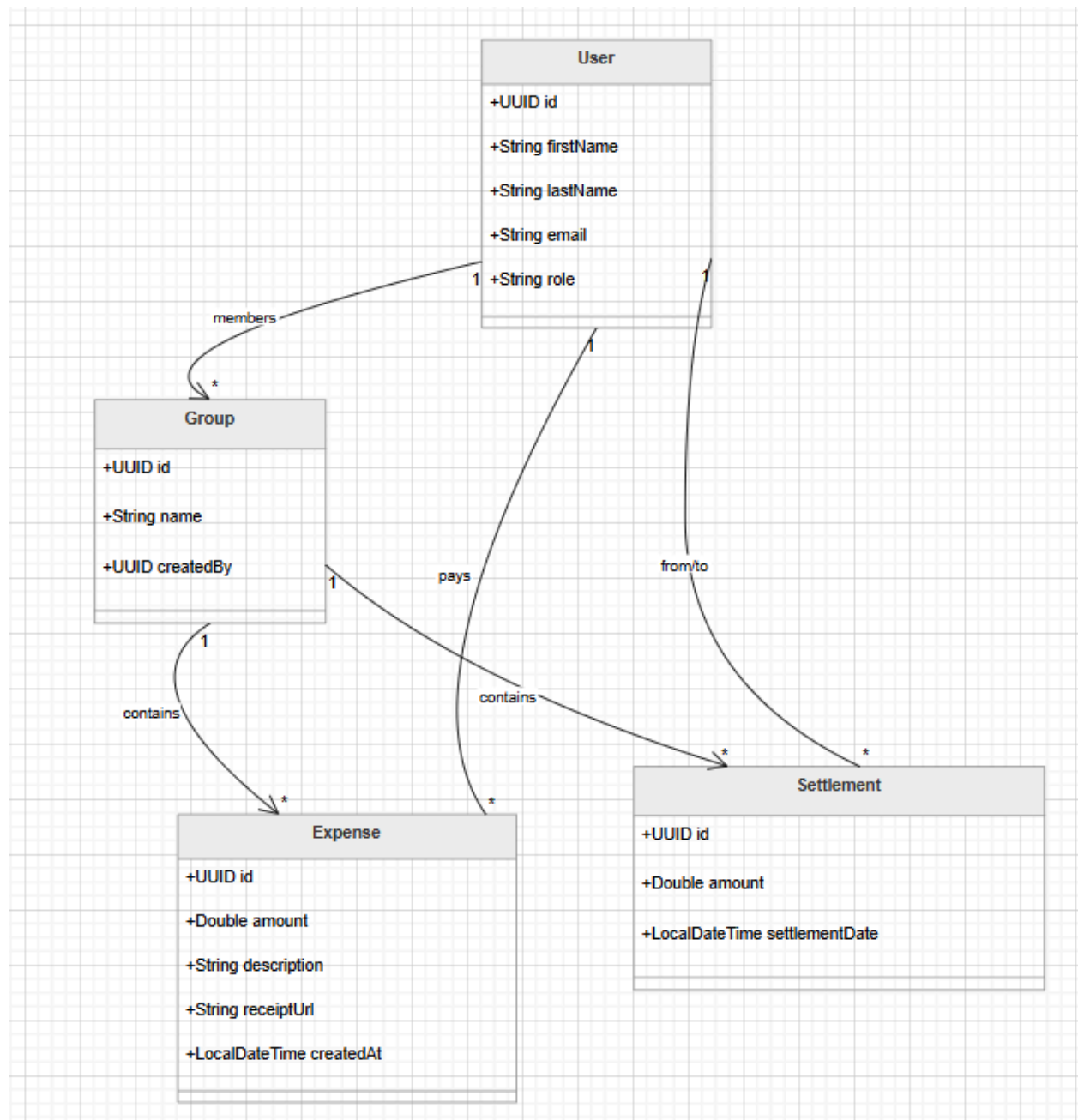
- **Error Handling:** Clear error messages with recovery options (e.g., "Image too large. Please upload a file smaller than 2MB" or "Insufficient permissions to delete this expense").
- **Mobile Ergonomics:** Touch targets for all primary actions (Add Expense, Settle Up, Camera/Upload) must be a minimum of **44x44px** to prevent input errors on mobile devices.
- **Keyboard Support:** Full keyboard navigation support for the web dashboard, allowing users to log expenses using tab-indexing and shortcuts.

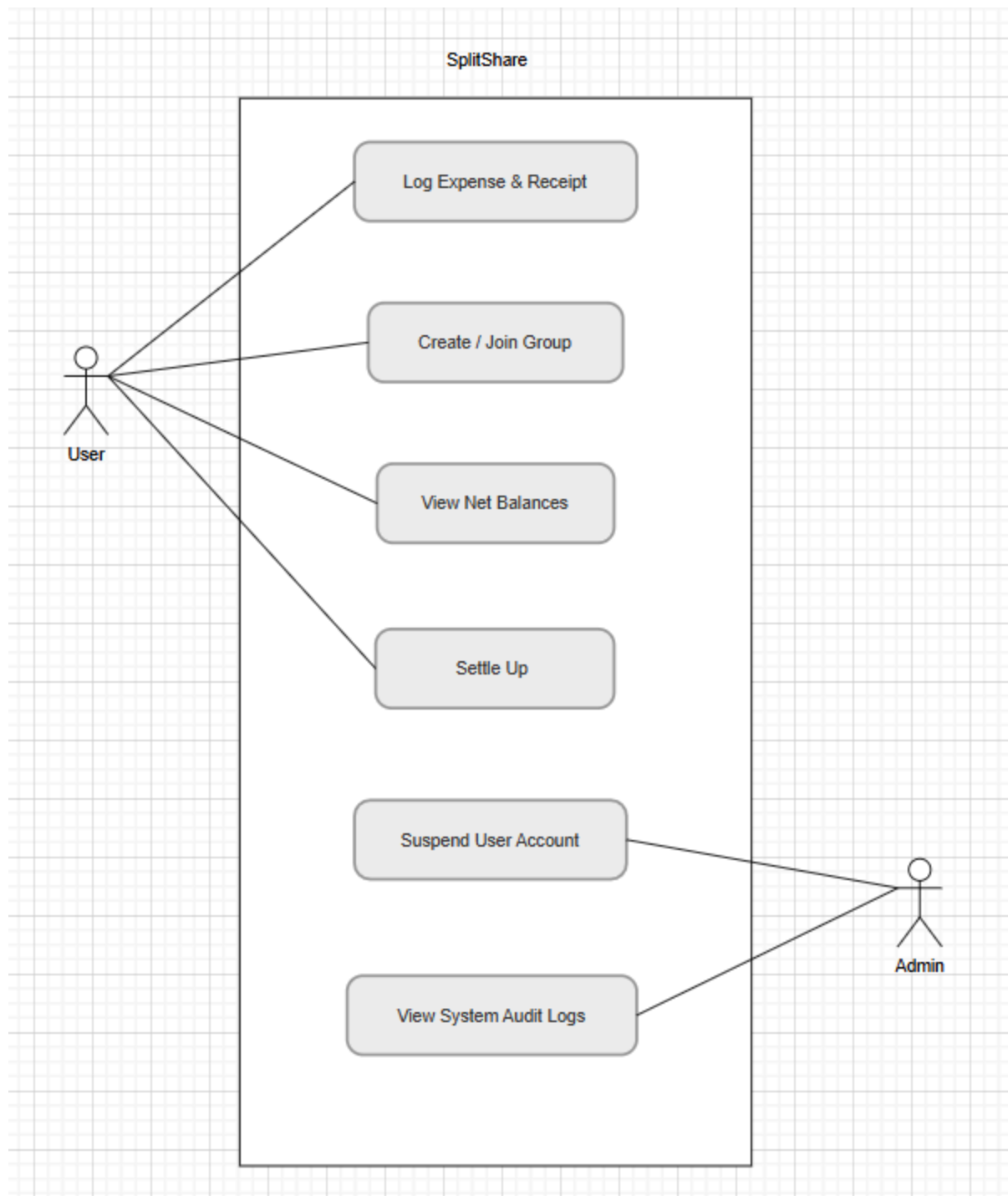
4.0 SYSTEM ARCHITECTURE

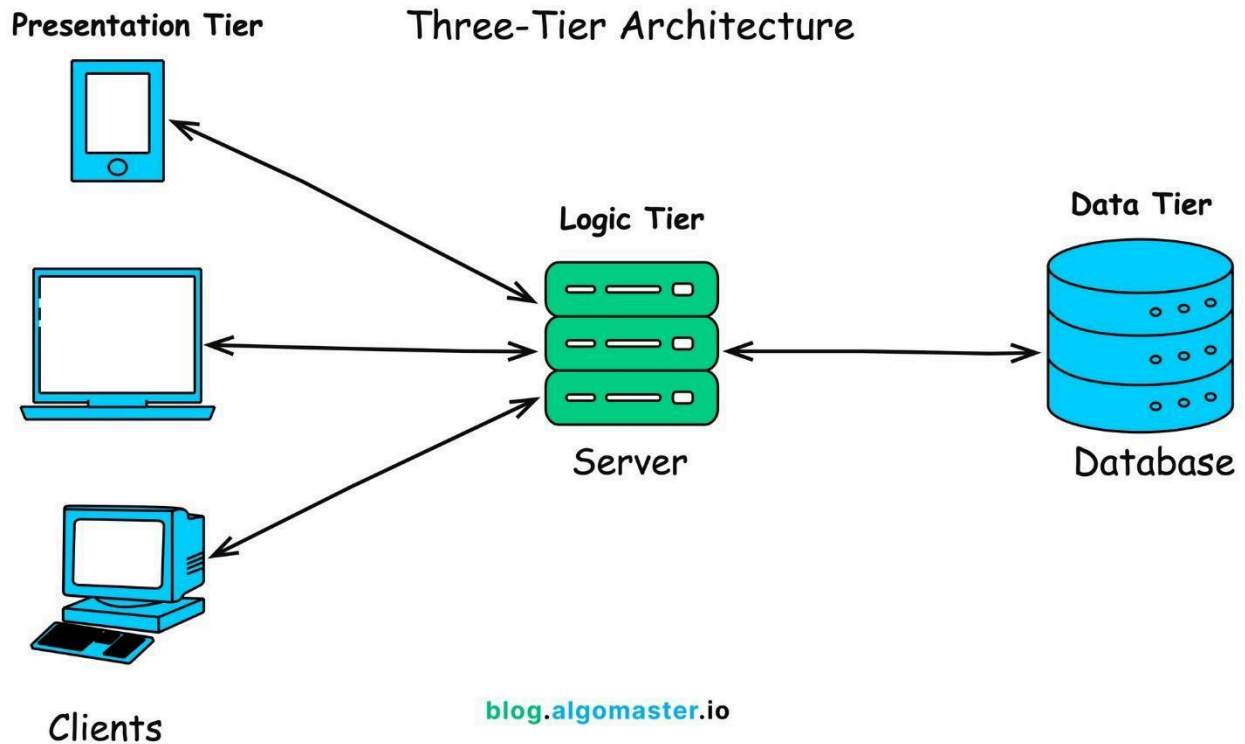
4.1 Component Diagram

Note: This should be a component diagram









Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security (JWT), Spring Data JPA.
- **Database:** Supabase (PostgreSQL, Auth, Storage, Edge Functions)
- **Web Frontend:** React 19, TypeScript, Tailwind CSS.
- **Mobile:** Kotlin, Jetpack Compose, Retrofit (API communication).

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

User Sign-up	<code>https://[server_hostname]:[port]/api/v1</code>
Log Expense	JSON for all requests/responses
Settle Up	Bearer token (JWT) in Authorization header
Receipt Upload	<pre>{ "success": boolean, "data": object null, "error": { "code": string, "message": string, "details": object null }, }</pre>

	"timestamp": string }
--	--------------------------

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "email": <email>, "password": <password>, "firstname": <firstname>, "lastname": <lastname>, }
Response Structure	{ "success": boolean, "data": { user { "email": <email>, "firstname": <firstname>, "lastname": <lastname>, }, "accessToken": <token>, "refreshToken": <token>, }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	<pre>{ "email": <email>, "password": <password>, }</pre>
Response Structure	<pre>{ "success": boolean, "data": { "user": { "email": <email>, "firstname": <firstname>, "lastname": <lastname>, "role": <role>, }, "accessToken": <token>, "refreshToken": <token>, }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	<pre>json { "success": false, "data": null, "error": {</pre>
-----------	--

	<pre> "code": "AUTH-001", "message": "Invalid credentials", "details": "Email or password is incorrect" }, "timestamp": "2024-01-28T10:30:00Z" } </pre>
Example 2	<pre> { "success": false, "data": null, "error": { "code": "VALID-001", "message": "Validation failed", "details": { "email": "Email is required", "password": "Must be at least 8 characters" } }, "timestamp": "2024-01-28T10:30:00Z" } </pre>

Common Error Codes

- **AUTH-001:** Invalid credentials
- **AUTH-002:** Token expired
- **GROUP-001:** User is not a member of this group
- **DB-001:** Transaction or Expense not found
- **VALID-001:** Invalid amount (e.g., negative value or zero)
- **SYSTEM-001:** Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)

Detailed Relationships:

- **Many-to-Many:** Users ↔ Groups (Managed via group_members join table).
- **One-to-Many:** Groups → Expenses (A group can have multiple shared costs).
- **One-to-Many:** Expenses → Expense Splits (One expense is divided among multiple users).

- **One-to-Many:** Groups → Settlements (A group has many individual "Settle Up" transactions).
- **Many-to-One:** Expenses → Users (Each expense is paid by one specific user).

Key Tables:

Table Name	Description
users	Stores core account details and authentication roles.
groups	Represents a shared space (e.g., "Roommates" or "Trip").
group_members	A join table that connects users to the groups they belong to.
expenses	Records the main transaction details (who paid and how much).
settlements	Records of manual "Settle Up" actions (The "Debt" clearer).

Table Structure Summary:

1. Table: users

- id: **BIGINT** (Primary Key, auto-increment)
- first_name: **VARCHAR** (Required)
- last_name: **VARCHAR** (Required)
- email: **VARCHAR** (Unique/Required)
- password: **VARCHAR** (Hashed via BCrypt)
- role: **VARCHAR** (e.g., 'ROLE_USER', 'ROLE_ADMIN')
- enabled: **BOOLEAN** (Default: true — set to false when an admin suspends the account)
- currency: **VARCHAR** (Nullable — the user's preferred display currency)
- created_at: **TIMESTAMP**

2. Table: groups

- id: **BIGINT** (Primary Key, auto-increment)
- name: **VARCHAR** (Required)
- created_by: **BIGINT** (Foreign Key to users.id)
- created_at: **TIMESTAMP**

3. Table: group_members

- id: **BIGINT** (Primary Key, auto-increment)
- group_id: **BIGINT** (Foreign Key to groups.id)
- user_id: **BIGINT** (Foreign Key to users.id)

4. Table: expenses

- id: **BIGINT** (Primary Key, auto-increment)
- group_id: **BIGINT** (Foreign Key to groups.id)
- paid_by: **BIGINT** (Foreign Key to users.id, NOT NULL)
- description: **VARCHAR**
- category: **VARCHAR** (NOT NULL — e.g., Food, Rent, Transport)
- amount: **NUMERIC**(10, 2) (Precision for PHP currency)
- receipt_url: **VARCHAR**
- created_at: **TIMESTAMP**

5. Table: balance_settlements

- id: **BIGINT** (Primary Key, auto-increment)
- group_id: **BIGINT** (Foreign Key to groups.id, NOT NULL)
- user_a_id: **BIGINT** (Foreign Key to users.id, NOT NULL)
- user_b_id: **BIGINT** (Foreign Key to users.id, NOT NULL)
- amount: **NUMERIC**(10, 2) (NOT NULL)
- confirmed_by_user_a: **BOOLEAN** (Default: false)
- confirmed_by_user_b: **BOOLEAN** (Default: false)
- settled_at: **TIMESTAMP** (Nullable — set when both users confirm)
- created_at: **TIMESTAMP** (NOT NULL)

6. Table: user_follows

- id: **BIGINT** (Primary Key, auto-increment)
- follower_id: **BIGINT** (Foreign Key to users.id, NOT NULL)
- following_id: **BIGINT** (Foreign Key to users.id, NOT NULL)
- created_at: **TIMESTAMP** (NOT NULL)
- UNIQUE constraint on (follower_id, following_id)

7. Table: admin_audit_logs

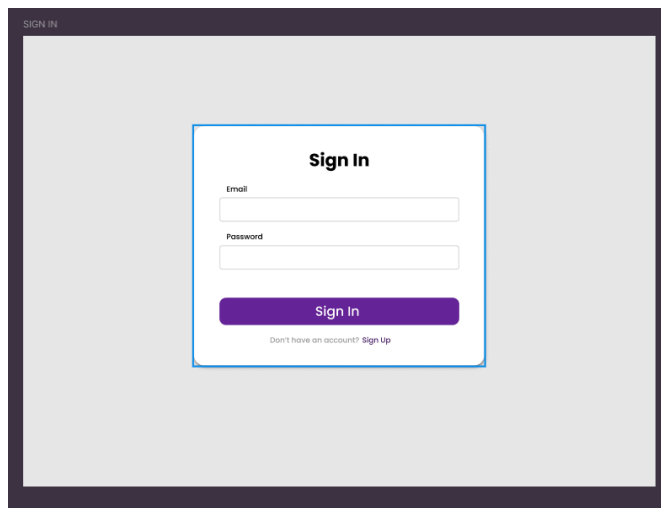
- id: **BIGINT** (Primary Key, auto-increment)
- action: **VARCHAR**(64) (NOT NULL — e.g., SUSPEND_USER, REACTIVATE_USER)
- actor_email: **VARCHAR** (NOT NULL — the admin who performed the action)
- target_user_id: **BIGINT** (Nullable)
- target_user_email: **VARCHAR** (Nullable)

- details: **VARCHAR**(1000) (Nullable)
- created_at: **TIMESTAMP** (NOT NULL)

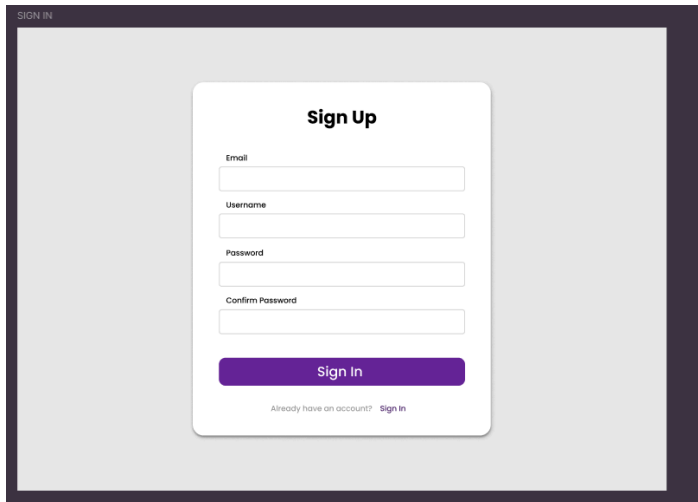
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma

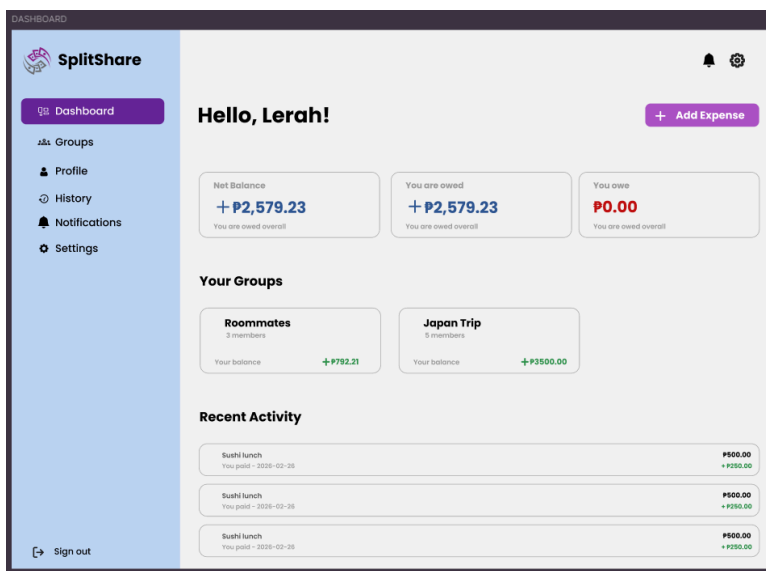


Description: This screen serves as the initial entry point for existing users. It features a clean, minimalist card interface centered on the screen, containing input fields for Email and Password. A prominent purple "Sign In" button initiates the authentication process via Supabase. A "Sign Up" link is provided at the bottom for new users to navigate to the registration flow.



The image shows a 'Sign Up' form centered on a dark background. The form is a white card with the title 'Sign Up' at the top. It contains four input fields: 'Email', 'Username', 'Password', and 'Confirm Password'. Below these fields is a purple 'Sign in' button. At the bottom of the card, there is a link that says 'Already have an account? Sign in'.

Description: The registration interface allows new users to create an account by providing an Email, Username, and Password (with a confirmation field). The design maintains consistency with the Sign In page, utilizing a centered white card against a dark background. Upon clicking the purple action button, a new user profile is generated in the database.



The image shows a user dashboard for 'SplitShare'. The left sidebar contains navigation links: Dashboard (selected), Groups, Profile, History, Notifications, and Settings. The main content area is titled 'Hello, Lerah!' and includes an 'Add Expense' button. It features three summary cards: 'Net Balance' showing +P2,579.23, 'You are owed' showing +P2,579.23, and 'You owe' showing P0.00. Below these are 'Your Groups' cards for 'Roommates' and 'Japan Trip'. At the bottom is a 'Recent Activity' table with three entries for 'Sushi lunch'.

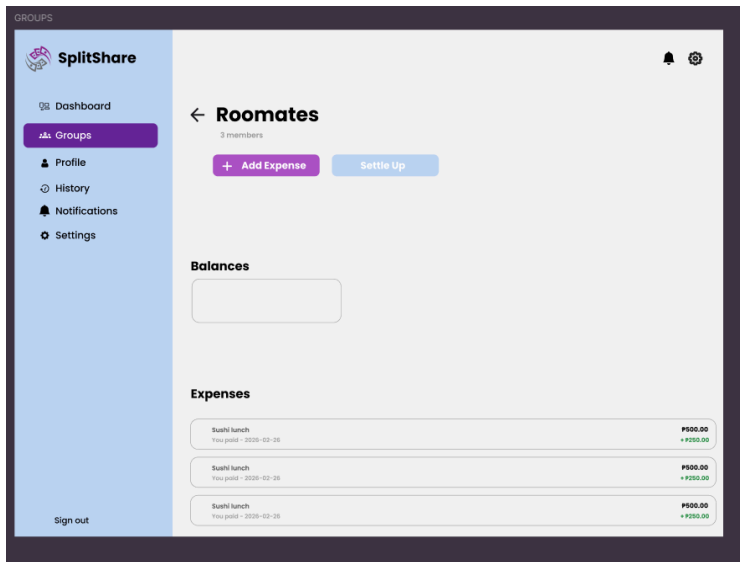
Activity	Date	Amount
Sushi lunch	You paid - 2025-02-25	P500.00 + P250.00
Sushi lunch	You paid - 2025-02-25	P500.00 + P250.00
Sushi lunch	You paid - 2025-02-25	P500.00 + P250.00

Description: After a successful login, users are greeted with a personalized dashboard. The left sidebar provides navigation to Groups, Profile, History, and Settings. The main content area features three summary cards:

- **Net Balance:** An overall total of what the user is owed versus what they owe.

- **You are owed:** Total amount currently due to the user.
- **You owe:** Total amount the user needs to pay back.

Below the summaries, the "Your Groups" section displays active group cards (e.g., Roommates, Japan Trip) with real-time balance previews. The "Recent Activity" feed at the bottom provides a chronological list of the latest transactions.

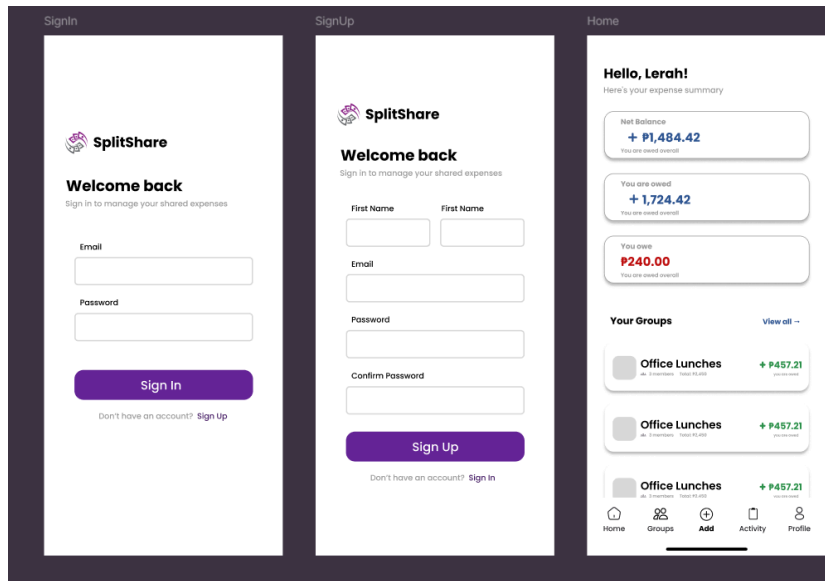


Description: This screen is accessed by selecting a specific group. It displays the group name ("Roommates") and the number of members. Primary action buttons for "Add Expense" and "Settle Up" are located at the top for quick access.

The "Balances" section is designed to show a breakdown of individual member statuses, while the "Expenses" section provides a detailed, scrollable list of all financial entries specific to this group, including dates, total amounts, and the user's specific share of that expense.

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma



Mobile-Specific Features

- **Touch-Optimized Interaction:** All critical action buttons (e.g., "Add Expense," "Settle Up") follow a minimum **44x44px** hit area to prevent accidental clicks on mobile devices.
- **Gesture Support:** Implementation of **swipe-to-delete** for expense drafts and **pull-to-refresh** on the group dashboard to fetch the latest balance updates from Supabase.
- **Offline Data Caching:** Use of **Room Database** (Android) to cache the "Net Balance" and recent transaction history, allowing users to check what they owe even without an active internet connection.
- **Bottom Navigation:** Primary navigation bar at the bottom of the screen for easy one-handed access to *Home*, *Groups*, *Add (+)*, and *Profile*.
- **Simplified Inputs:** Mobile-optimized forms using numeric-only keyboards for amount entries and native date pickers to reduce typing effort.

Design System:

- **Colors (Lavender Theme):** * **Primary:** #9D7BFF (Main Brand/Action Color)
- **Secondary:** #F3E8FF (Backgrounds/Card Accents)
- **Success:** #10B981 (Owed to you / Settlement confirmed)
- **Error:** #EF4444 (You owe / Delete actions)
- **Neutral:** #1F2937 (Text and Headings)

Typography: * **Font Family:** *Inter* (Sans-serif) for high legibility on small screens.

- **Sizing:** Responsive scale using rem (Web) and sp (Android) to ensure accessibility.

Spacing: Consistent **8px grid system** used for padding, margins, and component alignment to maintain a clean, organized layout.

Components: * **Cards:** Elevation-based cards for individual groups and summary balances.

- **Modals:** Bottom-sheet modals for "Settle Up" confirmations on mobile.
- **Inputs:** Outlined text fields with clear validation states for PHP (₱) currency formatting.

Responsiveness: * **Mobile-First Approach:** Layouts are designed for 360px width first.

- **Breakpoints:** * Mobile: < 640px
 - Tablet: 640px - 1024px
 - Desktop: > 1024px

8.0 PLAN

8.1 Project Timeline

This project is structured over a 10-week period, focusing on a "Backend-First" approach followed by cross-platform frontend integration.

Phase 1: Planning & Design (Weeks 1-2)

- **Week 1:** Requirements Gathering & UI/UX Wireframing (Figma).
- **Week 2:** System Architecture design and Supabase schema configuration.

Phase 2: Database & Backend Foundation (Weeks 3-4)

- **Week 3:** Setup Supabase project, Auth policies, and Storage buckets for receipts.
- **Week 4:** Develop Spring Boot REST API for core business logic (Balance calculations and group management).

Phase 3: Web Application Development (Weeks 5-6)

- **Week 5:** React project setup and integration with Supabase Auth.
- **Week 6:** Implementation of Group Dashboards, Expense Feed, and Settle Up functionality for Web.

Phase 4: Mobile Application Development (Weeks 7-8)

- **Week 7:** Android Studio setup (Kotlin/Jetpack Compose) and API service layer.
- **Week 8:** Mobile-specific features: Camera integration for Receipt Upload and push notifications.

Phase 5: Integration, Testing & Deployment (Weeks 9-10)

- **Week 9:** End-to-end testing (ensuring Web and Mobile reflect the same data in real-time).
- **Week 10:** Final Bug fixes and Deployment (Vercel for Web, Railway for Backend).

8.2 Milestones

Milestone	Description	Target Date
M1: Design Approval	All wireframes and SDD sections completed.	[Date]
M2: Data Layer Ready	Supabase tables, RLS policies, and Auth are fully functional.	[Date]
M3: API Functional	Spring Boot backend successfully handles financial logic.	[Date]
M4: Web MVP	Users can log expenses and view balances via browser.	[Date]

M5: Mobile MVP	Users can upload receipts.	[Date]
M6: System Baseline	Final integrated system ready for submission.	[Date]

8.3 Risk Mitigation

- **Data Inconsistency:** To prevent "phantom" balances, all expense entries and settlements will be handled as atomic transactions in PostgreSQL.
- **Security Risks:** Supabase Row Level Security (RLS) will be strictly enforced to prevent User A from seeing User B's private group data.
- **Scope Creep:** The "Must-Have" features (Auth, Log Expense, Settle Up) will be prioritized before adding "Could-Have" features like spending charts.